

CAIET DE PRACTICĂ

Proiect: Aplicație ride-hailing bazată pe microservicii

Nume și prenume student: Koma Kristian-Nicolae

Locul de desfășurare a practicii: Universitatea Tehnică din Cluj - Napoca Centrul Universitar Nord din Baia Mare

An/Specializarea/Grupă: CAL 2025-2026

Cadrul didactic supervisor de practică: Petrovan Adrian.

1. Introducere

Motivul alegerii proiectului. Majoritatea platformelor și aplicațiilor moderne funcționează pe sisteme bazate pe microservicii, astfel încât, dacă un serviciu cade, întregul ansamblu trebuie să continue să funcționeze normal, iar pentru a înțelege în profunzime acest principiu am ales să dezvolt în cadrul stagiului de practică o aplicație de tip ride-hailing construită integral pe o astfel de arhitectură.

Scopul proiectului. Realizarea unei platforme de tip ride-hailing complet funcționale, bazată pe microservicii, prin care utilizatorii să poată solicita și urmări curse în timp real, iar șoferii să le accepte prin intermediul unei aplicații dedicate. Platforma trebuie să fie scalabilă și rezistentă la defecțiuni: fiecare serviciu are o responsabilitate clară, comunică cu celelalte prin protocoale standardizate, iar căderea unui singur serviciu nu trebuie să afecteze întreaga aplicație.

Descrierea proiectului. Utilizatorul selectează direct pe hartă punctul de preluare și destinația cursei, pe baza cărora serviciul de prețuri calculează tariful în funcție de distanță. După acceptarea prețului, serviciul de dispatch găsește cel mai potrivit șofer disponibil (cel mai apropiat de locația clientului), iar oferta este transmisă în timp real șoferului prin WebSocket; acceptarea acestuia finalizează asignarea cursei. Toate fluxurile sunt orchestrate de un serviciu central (Coordonatorul), care face legătura dintre frontend și restul microserviciilor.

Proiectul este organizat ca monorepo și cuprinde: trei servicii backend (coordonator, dispatch, prețuri) dezvoltate cu NestJS, două aplicații frontend realizate cu Svelte 5 (una pentru clienți, una pentru șoferi), un pachet de tipuri și componente partajate, precum și definiții protobuf pentru comunicarea gRPC. Comunicarea între servicii folosește atât gRPC (ConnectRPC) cât și mesaje asincrone prin RabbitMQ, iar infrastructura (RabbitMQ, Redis, PostgreSQL) este orchestrată cu Docker Compose.

2. Activități realizate în cadrul stagiului de practică

Săptămâna 1 (16.07–23.07.2026)

- Creat board Trello pentru organizarea sarcinilor și pagina de GitHub a proiectului.
- Creat template-ul pentru frontend folosind Svelte 5.
- Adăugat cerere de permisiune pentru geolocație în scopul aflării locației exacte a utilizatorului.
- Implementat logica de waypoint: punct de plecare și punct de sosire; la alegerea celor două puncte se deschide un modal care solicită calcularea prețului și găsirea unui șofer.
- Inițializat serviciul Dispatch; analizate variantele de stack pentru backend și adăugată librăria ConnectRPC (gRPC) pentru comunicarea client–server.
- Inițializat serviciul Price cu logică de calculare a prețului pe baza distanței.
- Inițializată aplicația FrontendDriver (aplicație dedicată șoferilor), cu adăugarea poziției și configurare pentru ConnectRPC.
- Restructurat repository-ul într-un monorepo și adăugat fișierul proto pentru comunicarea gRPC.
- Implementată logica de RPC pentru driver (client și server) și buton de marcarea șoferului ca fiind activ.
- Realizată cu succes comunicarea între frontend și backend; începută diagrama arhitecturală.
- Inițializat serviciul Coordonator, adăugată comunicarea RPC între microservicii (Price) și direcționarea tuturor cererilor frontend-ului prin Coordonator.

Săptămâna 2 (27.07–3.08.2026)

- Revizuit planul de arhitectură: renunțat la Kafka (considerat overkill) în favoarea RabbitMQ pentru comunicare asincronă și pg-boss pentru cron jobs; adoptată arhitectura orchestrată, cu Coordonatorul ca “central brain”.
- Adăugat RabbitMQ cu docker-compose și implementată comunicarea asincronă între backend (canale de queue), în serviciile Dispatch și Coordonator.
- Migrat la o versiune mai nouă de RabbitMQ, cu opțiune de retry pentru mesajele respinse; reparate exchange-key-urile greșite.
- Adăugat suport WebSocket în Dispatch; stabilită comunicarea prin WebSocket cu FrontendDriver și realizată comunicarea SSE între Coordonator și frontend, după analiza limitărilor de streaming ale ConnectRPC în browsere.
- Crearea modalului de ofertare a cursei pentru șoferi, cu butoane de acceptare/respingere; reparate URL-urile SSE și WebSocket (socket.io folosește HTTP și face upgrade ulterior).
- Implementată logica de găsim a celui mai bun șofer disponibil (cel mai apropiat de punctul de plecare).
- Adăugat Redis pentru pub/sub între mai multe instanțe active (Coordonator și Dispatch), cu modul de inițializare Redis și migrarea logicii din memory map în Redis.

- Implementate lock-uri pentru șoferi prin Redis, pentru idempotență și prevenirea corupției datelor; reparată eroarea de routing-key la mesajele de failure.
- Refactorizat frontend-ul: SSE mutat într-un store propriu cu logică event-driven, store-uri pentru ride/driver și fișier de tipuri; în FrontendDriver, mapă de stări pentru butoane.
- Adăugat pasul “DriverFound” (afișarea informațiilor despre cursă) și separată logica repetată într-un folder utils.
- Începută integrarea PostgreSQL cu Drizzle (ORM) în Dispatch, fără a fi încă utilizată.

Săptămâna 3 (06.08–13.08.2026)

- Creat pachete partajate în monorepo: @goto/domain (tipuri comune: LatLng, RideLocations, RideOffer), @goto/map (componentă de hartă reutilizabilă) și @goto/proto (definiții protobuf cu generare de cod prin buf, atât pentru Connect cât și pentru NestJS).
- Integrat complet PostgreSQL + Drizzle: scheme pentru curse (rides), șoferi (drivers), utilizatori (users) și outbox, cu migrații și repository-uri dedicate.
- Implementat outbox pattern în Coordonator pentru publicarea fiabilă a evenimentelor (stări pending/published/failed, utilizarea SKIP LOCKED și curățarea periodică a evenimentelor publicate).
- Mutată logica de ride a Dispatch pe PostgreSQL; adăugate cron jobs (@nestjs/schedule, la fiecare 5 secunde) pentru asignarea curselor aflate în starea pending, notificarea curselor anulate și curățarea ofertelor expirate.
- Frontend: adăugat store de sesiune, modale de finalizare/eșuare a cursei (DriveComplete, RideFailed), câmp pentru identificator utilizator și îmbunătățirea afișării markerilor șoferilor.
- FrontendDriver: decuplată logica WebSocket în store-uri proprii, implementate stări distincte ale butoanelor pentru acțiunile șoferului și afișarea poziției clientului.
- Coordonator: adăugat modulul dispatchgrpc pentru streaming-ul poziției șoferilor, mutarea logicii din controller în service și interceptor de logare.
- Extins docker-compose cu PostgreSQL 18 + pgAdmin și Redis Insight pentru dezvoltare și monitorizare.

3. Concluzii și abilități dobândite

- **NestJS (backend):** structurarea aplicațiilor pe module, dependency injection, separarea logicii în controllere și servicii, gateway-uri WebSocket, lifecycle hooks și orchestrarea mai multor microservicii.
- **Arhitectura de microservicii:** partiționarea aplicației pe servicii independente (coordonator, dispatch, prețuri), arhitectură orchestrată cu un “central brain” și analiza comparativă a soluțiilor de comunicare (Kafka vs. RabbitMQ).
- **gRPC și ConnectRPC:** definirea contractelor de comunicare în protobuf, generarea automată de cod cu buf pentru mai multe limbaje/framework-uri și realizarea de stream-uri bidirecționale între servicii.
- **RabbitMQ:** comunicare asincronă prin exchanges, routing keys și queues, stabilirea canalelor de mesagerie între servicii, retry la mesajele respinse și debugging-ul legăturilor greșite.
- **Redis:** pub/sub între multiple instanțe active și lock-uri distribuite pentru a obține idempotență și a preveni corupția datelor în operații concurente.
- **PostgreSQL și Drizzle ORM:** modelarea schemei (curse, șoferi, utilizatori), migrații, repository-uri, tranzacții cu SKIP LOCKED și implementarea outbox pattern pentru publicarea fiabilă a evenimentelor.
- **Cron jobs și task scheduling:** programarea sarcinilor periodice cu @nestjs/schedule (asignarea curselor, notificarea curselor anulate, curățarea ofertelor expirate) și înțelegerea alternativelor precum pg-boss.
- **Comunicare în timp real:** WebSocket cu socket.io pentru streaming bidirecțional către șoferi și Server-Sent Events (SSE) pentru notificări unidirecționale către client.
- **Svelte 5 și SvelteKit:** aplicații component-based cu runes și store-uri, logică event-driven pentru decuplarea componentelor, gestionarea stărilor de ride/driver și integrarea cu API-uri și stream-uri.
- **Hărți și geolocație:** utilizarea Leaflet pentru afișarea hărților, markerilor și rutelor, precum și obținerea poziției utilizatorului prin geolocație.
- **Frontend modern:** TanStack Query pentru mutations și stare server, Tailwind CSS cu shadcn-svelte pentru interfață și modalele de flux (ofertă, finalizare, eșuare cursă).
- **Monorepo și instrumente:** lucrul cu pnpm workspaces și pachete partajate (@goto/domain, @goto/map, @goto/proto), versionare cu Git/GitHub și organizarea sarcinilor cu Trello.
- **Docker și Docker Compose:** orchestrarea infrastructurii locale, RabbitMQ, Redis, PostgreSQL, pgAdmin și Redis Insight, cu volume persistente și healthchecks.
- **Inginerie software:** debugging-ul sistemelor distribuite, testare (Jest, E2E, Playwright), refactorizare și curățarea codului, precum și luarea deciziilor arhitecturale motivate.

În concluzie, stagiul de practică mi-a oferit o experiență completă de dezvoltare a unei aplicații distribuite, de la alegerea tehnologiilor și proiectarea arhitecturii, până la implementare, integrare și depanare.